

PLP - Foundation Match BR (Rascunho + Requisitos)

2026-09-17 · @Someone

Contexto — o que foi pedido para 21/09

Atividade do Classroom **"Explorar pilares do Paradigma Orientado e Objeto"** (disciplina CC.P5.447 — Paradigmas de Linguagens de Programação), postada por Emmanoel Monteiro de Sousa Junior em 14/09, vale 2 pontos.

Enunciado: expressar os paradigmas de **Abstração, Herança, Polimorfismo e Encapsulamento** em três linguagens distintas à escolha do aluno. Apresentar as soluções em sala e enviar os arquivos após a avaliação presencial, para evidenciar a atividade.

Prazo: segunda-feira, 21/09, 23:59.

Suposição assumida (registrando para vocês validarem): a entrega de segunda é código pequeno e direto, demonstrando os 4 pilares em 3 linguagens — não o site completo. Vamos usar o domínio do projeto do site (produto "Base/Foundation") como estudo de caso dos exemplos, porque isso já serve de rascunho para o projeto maior e evita retrabalho.

Ideia do projeto — referência Findation.com

O Findation funciona como um "tradutor de tom entre marcas": a pessoa informa as bases que já usa e que combinam com sua pele (ex.: MAC NC30), e o site indica o tom equivalente em outras marcas (Fenty, Revlon, L'Oréal etc.), usando uma base de mais de 11 mil produtos e 106 mil tons cadastrados. O cadastro é feito via login do Google, e o site orienta o usuário a informar só tons que realmente deram certo, para não distorcer a recomendação. Também existe uma página voltada a lojistas/varejistas.

Proposta do grupo: um site nos mesmos moldes, mas focado no mercado brasileiro (marcas vendidas no Brasil, preços em real, onde comprar aqui), com nome provisório a definir — sugestão: **"TomCerto"** (ajustar se preferirem outro).

Entrega de 21/09 — os 4 pilares em 3 linguagens

Modelo de domínio único, implementado nas 3 linguagens, para deixar claro que o paradigma é o mesmo — só a sintaxe muda.

Linguagens sugeridas: Python, Java e TypeScript (ou JavaScript). Critério: são as que vocês já têm mais familiaridade e cobrem estilos diferentes (tipagem dinâmica, estática, e um meio-termo).

Modelo de domínio (mesma estrutura nas 3 linguagens):

1. **Abstração** — classe abstrata `Produto` (ou `Base`), com atributos comuns (`marca`, `nome`, `preco`) e um método abstrato `calcularCompatibilidade(tomPele)`, sem implementação na classe-mãe.
2. **Herança** — subclasses `BaseLiquida`, `BaseCushion`, `BaseMousse` herdando de `Produto`, cada uma acrescentando atributos próprios (ex.: `fps` na líquida, `numeroEsponjas` no cushion).
3. **Polimorfismo** — cada subclasse implementa `calcularCompatibilidade()` do seu próprio jeito; um mesmo laço percorrendo uma lista de `Produto` chama o método certo para cada tipo, sem `if/else` de tipo.
4. **Encapsulamento** — atributos privados/protegidos (ex.: `_tomBase`, `#preco`) acessados só por getters/setters, com validação simples (ex.: preço não pode ser negativo).

Esse mesmo modelo vira, depois, a base das classes reais do backend — não é código descartável.

Formato de entrega: uma pasta por linguagem (ver estrutura de diretórios abaixo), com um `README.md` curto explicando onde está cada pilar no código, para facilitar a apresentação em sala.

Documento de requisitos

Requisitos funcionais base (equivalentes ao Findation)

Cód.	Requisito	Descrição
RF01	Cadastro/login	Login por e-mail/senha ou Google
RF02	Cadastro de tons já usados	Usuário informa marca + tom de bases que já usou e que combinaram
RF03	Motor de compatibilidade	A partir dos tons informados, sugerir tons equivalentes em outras marcas
RF04	Catálogo de produtos	Base de marcas, produtos e tons (nome, marca, tom, preço)
RF05	Busca/filtro	Filtrar por marca, faixa de preço, tipo de pele

Cód.	Requisito	Descrição
RF06	Página do produto	Detalhes do produto + tons disponíveis

Melhorias propostas pelo grupo

Cód.	Requisito	Descrição
RF07	Seleção de subtom	Usuário escolhe subtom (frio, quente, neutro, oliva) além do tom
RF08	Guia de subtom	Quiz com opções pré-definidas: aparência das veias sob luz solar, reação da pele à exposição solar, teste do papel branco e cor natural dos lábios — indica se o subtom é frio, quente, neutro ou oliva
RF09	Preferência de textura	Usuário escolhe o acabamento que procura ou está buscando: matte, semi-matte, glow/luminoso, entre outros
RF10	Preferência de espessura/formato	Usuário escolhe a fórmula que prefere: líquida, fluida, cushion, mousse, entre outras
RF11	Avaliações da comunidade	Usuário se cadastra para avaliar bases/corretivos, informando há quanto tempo usa e há quanto tempo comprou o produto; pode optar por deixar visível, para outras pessoas, o tom, tipo (ex.: oleosa) e subtom (ex.: oliva) da própria pele
RF12	Onde comprar no Brasil	Lista de lojas/e-commerces nacionais que vendem aquele produto, com link e preço quando disponível
RF13	Perfil de pele salvo	Usuário salva tom, subtom, textura e tipo de pele preferidos para receber recomendações sem repetir a busca
RF14	Comparador de produtos	Comparar 2-3 bases lado a lado (tom, textura, preço, avaliações)

Detalhamento das melhorias (RF07-RF12)

RF07/RF08 — Subtom e guia de subtom

- Campo de subtom: frio, quente, neutro, oliva.
- Quiz de apoio para quem não sabe o subtom, com perguntas de múltipla escolha pré-definidas:
 1. Aparência das veias do pulso sob luz solar (azuladas/arroxeadas → frio; esverdeadas → quente; mistura → neutro).
 2. Reação da pele à exposição solar (queima com facilidade vs. bronzeia com facilidade).
 3. Teste do papel branco (a pele parece amarelada, rosada ou acinzentada perto de um papel branco).
 4. Cor natural dos lábios (tons nude/rosados → frio; coral/pêssego → quente).
- O resultado do quiz preenche automaticamente o campo de subtom do perfil salvo (RF13).

RF09/RF10 — Textura e formato

- RF09 (acabamento): usuário escolhe o que procura — matte, semi-matte, glow/luminoso — usado como filtro de busca e como preferência salva no perfil.
- RF10 (fórmula): usuário escolhe o formato preferido — líquida, fluida, cushion, mousse — também como filtro e preferência salva.
- Os dois campos são filtros combináveis com marca, preço e subtom.

RF11/RF12 — Avaliações e onde comprar

- Cada produto tem uma seção própria de avaliações e uma seção "onde comprar no Brasil".
- Para avaliar um produto, o usuário precisa estar cadastrado.
- Campos da avaliação: nota/comentário, há quanto tempo usa o produto, há quanto tempo o comprou.
- Campo opcional na avaliação: tornar visível para outras pessoas o tom, tipo de pele (ex.: oleosa) e subtom (ex.: oliva) de quem avaliou — ajuda quem tem pele parecida a decidir.
- "Onde comprar no Brasil": lista de lojas/e-commerces nacionais associada ao produto, com link e preço quando disponível.

Requisitos não funcionais

- Responsivo (desktop primeiro, mobile depois).
- Dados de avaliação de pele são opcionais e claramente marcados como tal (não travar o fluxo de quem não quiser informar).
- Tempo de resposta da busca/recomendação abaixo de 2s para uma base de dados de teste (não precisa escalar para milhões de produtos nesta fase acadêmica).

- Separação clara entre camada de dados (produtos, tons, avaliações), regra de negócio (motor de compatibilidade) e interface — é justamente onde os 4 pilares de POO se aplicam.

Fora de escopo por agora

- Integração de pagamento (o site indica onde comprar, não vende diretamente).
- App mobile nativo (fica no plano de expansão).

Estrutura de diretórios

Para a entrega de 21/09 (repositório simples, sem necessidade de rodar servidor):

```

plp-tomcerto/
├── README.md                # o que é o projeto, como rodar cada exemplo
├── entrega-21-09/
│   ├── python/
│   │   ├── produto.py      # abstração + encapsulamento
│   │   ├── bases.py        # herança (BaseLiquida, BaseCushion,
BaseMousse)
│   │   ├── main.py         # demonstra o polimorfismo
│   │   └── README.md
│   └── java/
│       ├── Produto.java
│       ├── BaseLiquida.java
│       ├── BaseCushion.java
│       ├── BaseMousse.java
│       ├── Main.java
│       └── README.md
└── typescript/
    ├── produto.ts
    ├── bases.ts
    ├── main.ts
    └── README.md

```

Estrutura alvo do projeto completo (para quando a implementação do site começar de fato):

```

tomcerto/
├── docs/                                # este documento de requisitos e outros ADRs
├── backend/
│   ├── src/
│   │   ├── domain/                    # Produto, Base*, Avaliacao, Usuario, Loja
│   │   │   (POO do projeto)
│   │   ├── services/                  # motor de compatibilidade de tom/subtom
│   │   ├── repositories/              # acesso a dados (produtos, tons, avaliações)
│   │   └── api/                        # rotas/controllers
│   └── tests/
├── frontend/
│   ├── src/
│   │   ├── components/                # seletor de subtom, filtro de
│   │   │   textura/espessura, cards de avaliação
│   │   ├── pages/                     # home, busca, produto, comparador
│   │   └── services/                  # chamadas à API
│   └── tests/
└── database/
    ├── schema.sql                     # ou migrations, conforme a stack escolhida
    └── seeds/                         # dados de teste (marcas, produtos, tons)

```

A pasta `entrega-21-09/domain` pode, futuramente, ser promovida para `backend/src/domain` sem reescrever do zero — por isso vale a pena caprichar nela agora.

Divisão de tarefas — Eduarda e Natalia

Critério: cada uma fica dona de 1-2 linguagens completas (não de "metade de cada pilar"), para evitar dependência cruzada antes da apresentação.

Para a entrega de 21/09

Tarefa	Responsável
Python — <code>Produto</code> , <code>Base*</code> , <code>main.py</code> + README explicando os pilares	Eduarda
Java — <code>Produto</code> , <code>Base*</code> , <code>Main.java</code> + README explicando os pilares	Natalia
TypeScript — implementação + README	Dividir em par, ou quem tiver mais afinidade com a linguagem assume e a outra revisa
Revisão cruzada (cada uma lê o código da outra e confere se os 4 pilares aparecem claramente)	As duas, antes de segunda

Tarefa	Responsável
Preparar a fala da apresentação em sala (quem explica qual pilar)	As duas, dividindo os 4 pilares — 2 cada

Cronograma até 21/09

Quando	O quê
Qui 17/09 (hoje)	Fechar este rascunho e a divisão de tarefas
Sex 18/09	Cada uma implementa sua(s) linguagem(ns)
Sáb 19/09	Continuar implementação; começar os READMEs de cada pasta
Dom 20/09 (dia)	Terminar código; revisão cruzada
Dom 20/09 (noite)	Ensaiai a apresentação (quem fala qual pilar)
Seg 21/09	Apresentar em sala e enviar os arquivos até 23:59

Plano de expansão futura

Depois da entrega de POO, se decidirem continuar o projeto de verdade:

1. **MVP web** — backend com o domínio já criado (RF01-RF06) + frontend simples, banco com um catálogo pequeno digitado à mão (10–20 produtos) para validar o motor de compatibilidade.
2. **Melhorias do grupo** (RF07-RF14) — subtom, textura, espessura, avaliações com dados de pele, onde comprar no Brasil, comparador.
3. **Comunidade** — sistema de avaliações mais robusto (fotos antes/depois, filtro de avaliações por tom de pele parecido com o do usuário).
4. **Parcerias/monetização** — página para lojas brasileiras cadastrarem produtos e estoque (nos moldes da página de varejista do Findation), possível afiliação com links de compra.
5. **App mobile** — depois que a versão web estiver estável, considerar PWA ou app nativo.

Esse roadmap não faz parte da entrega de 21/09 — é o contexto que justifica por que vale a pena caprichar no modelo de domínio desde já.